CSCE 1030: Project 1
Due: 11:59 PM on Friday, April 10, 2025

PROGRAM DESCRIPTION

In this project, students will design and implement a console-based **Bank Account Simulator** in C++ that models a simplified banking system. Users create an account, select an account type—Checking, Savings, or Student—manage a balance, and perform PIN-protected deposits and withdrawals with robust input validation. The three account types embody distinct, realistic banking rules: a **Checking account** permits withdrawals beyond the current balance but applies a fixed overdraft fee; a **Savings account** allows withdrawals only when sufficient funds exist, preventing overdrafts entirely; and a **Student account** mirrors the conservative, low-risk behavior of Savings by disallowing overdrafts, making it suitable for beginner use. All transaction activity (deposits, withdrawals, fees) is recorded in fixed-size arrays and can be reviewed or filtered by type.

YOU CAN ONLY USE CONTENTS FROM Chapters 1-6 of the class materials to complete this program. If you do not follow this requirement, you will get a zero (0) for this assignment.

PROGRAM REQUIREMENTS

Banner

As with all projects in this course, your program's output will display your name, your EUID, your e-mail address, the department name, and course number. This means that your program will print this information to the terminal (see the sample output).

Build a console-based Bank Account Simulator that manages:

- Account holder name (letters/digits/spaces only)
- Account type: CHECKING, SAVINGS, or STUDENT (selected by entering 1–3)
- Balance (double)
- PIN (4–6 digits); required for Deposit/Withdraw
- **Transaction history (last 100 entries; arrays only):**
  - Type: 'D' (Deposit), 'W' (Withdraw), 'F' (Fee)
  - Amount (double)
  - Memo (string)

Note:

Transaction Storage Requirement:

Transaction history must be stored using **three parallel arrays**—one for the transaction type, one for the amount, and one for the memo—where all three arrays use the **same index** to represent a single transaction.

Consistency Rule:

When adding, removing, or shifting transactions (e.g., when the history is full), **all three arrays must be**

updated together so that the type, amount, and memo at a given index always belong to the same transaction.

Policy differences:

- Checking can overdraft: withdrawal allowed; a $35 fee is applied and recorded as 'F'.
- Savings/Student cannot overdraft: withdrawal that would go negative is denied (no changes recorded).

Required Features & Flow

1) Header— Print a banner
2) Name input — Read with getline; re-prompt until only letters/digits/spaces (checked with isalpha/isdigit/isspace).
3) Initial balance — Read a double.
4) Account type selection — Prompt for 1..3; validate with do/while; map to enum.
5) Main loop (inline menu + switch) — Display 6 options and handle the choice via switch:
   1) Deposit (PIN required)
   2) Withdraw (PIN required)
   3) Show Account Details
   4) Set/Change PIN (4–6 digits)
   5) Exit (with y/n confirmation)
   6) View Transactions by Type (D/W/F)
6) PIN enforcement — For Deposit/Withdraw, if no PIN exists, force creation (validate length & digits; confirm). If a PIN exists, prompt and compare; if wrong, cancel the operation.
7) Deposit — Amount (double) -> cin.get() -> memo via getline; increase balance and record 'D'.
8) Withdraw — Amount (double) -> cin.get() -> memo via getline; apply account-type rules (deny or overdraft with fee); record 'W' (and 'F' for fee).
9) Show Details — Print name, type, balance, and (set)/(not set) for PIN; show up to 10 most recent transactions (newest first).
10) View by Type — Ask for D/W/F (case-insensitive); print matching transactions or a message if none.

Function Specifications

1) void printHeader()
   - Print a banner (see sample output)

2) bool isValidName(const string& name)
   - Performs a character-by-character inspection of a string
   - Return true only if every character is is a letter, a digit, or a space.
   - Examples: 'Alex Johnson' -> true; 'Alex_Johnson' or 'Alex!' -> false.

3) string readValidName()
   - Read a line with getline and re-prompt until isValidName(...) is true.
   - On invalid input, print: Name can only contain letters, digits, and spaces. Try again:

- Return the validated name.

4) bool isValidPin(const string& p)
- Validates the security strength of a PIN.
- Return true only if length is 4-6 and all characters are digits.
- Examples: '1234' and '000000' -> true; '12', '1234567', '12a4' -> false.

5) bool setOrChangePin(string& pin, bool& pinSet)
- Manages PIN creation. It asks for a PIN, validates it, and then asks for a second "confirmation" entry to ensure they match.
- If match: set pin, set pinSet=true, print 'PIN set.' and return true.
- If mismatch: print 'PIN mismatch.' and return false.

6) bool requirePinForAction(string& pin, bool& pinSet)
- If no PIN: inform user and call setOrChangePin(...); return true on success, otherwise false.
- If PIN exists: prompt 'Enter PIN:'; if attempt != pin, print 'Incorrect PIN.' and return false; else return true.
- Used by Deposit and Withdraw.

7) AccountType chooseAccountType()
- Use do/while to re-prompt until a valid integer 1-3 is entered.
- switch(t): 1->CHECKING, 2->SAVINGS, 3->STUDENT; default prints error and repeats.
- Returns a valid enum value

8) string accountTypeToString(AccountType t)
- Map a valid enum to a label.
- Example: If the enum is STUDENT, this function returns the string "Student".

9) void recordTransaction(char type, double amount, const string& memo)
- If gTxnCount < MAX_TXN: append and increment.
- If full (100): shift-left (drop oldest) and write the new entry at index MAX_TXN-1.
- Accept type in {'D','W','F'}, amount >= 0, and any memo.

Adds a transaction to the history; if the array is full, removes the oldest transaction by shifting each stored transaction to the next lower index (index 1→0, 2→1, etc.) and stores the new transaction at the last index.
Example: If the history is full and a new $50 Deposit is made, Index 0 is deleted, Index 1 moves to 0, and the new $50 Deposit is saved at Index 99.

10) void printRecentTransactions(int maxToShow)
- If maxToShow <= 0, use 10. If gTxnCount == 0, print (No transactions yet).
- Otherwise print up to maxToShow from newest to oldest with labels:
  [Deposit] $amount | memo
  [Withdraw] $amount | memo

[Fee] $amount | memo

11) void viewTransactionsByType()
- It asks for a type (D/W/F) and scans the history, printing only the transactions that match that specific character. Prompt for D/W/F; accept lowercase by converting to uppercase; re-prompt until valid.
- Print only matching transactions from newest to oldest; if none, print (No transactions of that type).

12) void showDetails(const string& holder, AccountType type, double balance, bool pinSet)
- Generates a comprehensive account snapshot, including the holder's name, type, balance, and PIN status, followed by the transaction history.
- Print holder, accountTypeToString(type), balance (two decimals), and PIN status (set)/(not set).
- Call printRecentTransactions(10).

13) void deposit(double& balance, string& pin, bool& pinSet)
- Call requirePinForAction(...); if false, print 'Deposit canceled.' and return.
- Read amount; then call cin.get() and read memo with getline.
- Update balance += amount; recordTransaction('D', amount, memo); print new balance.

14) void withdraw(double& balance, AccountType type, string& pin, bool& pinSet)
- Call requirePinForAction(...); if false, print 'Withdrawal canceled.' and return.
- Read amount; call cin.get(); read memo with getline. Compute newBal = balance - amount.
- If newBal >= 0: set balance = newBal; record 'W'; print confirmation.
- If newBal < 0 and type == CHECKING: allow overdraft; apply $35 fee; set balance = newBal - 35.0; record 'W' then 'F' (memo 'Overdraft Fee'); print overdraft message.
- If newBal < 0 and type == SAVINGS or STUDENT: print 'Withdrawal denied.' with no changes.

INLINE MENU AND VALIDATION (IN MAIN)
- Display options 1-6 inline in main (no separate showMenu).
- Read choice with a do/while loop until 1-6, then switch(choice) to dispatch.
- On Exit, ask y/n and validate before leaving.

DESIGN (ALGORITHM):

On a piece of paper (or word processor), write down the algorithm, or sequence of steps, that you will use to solve the problem. You may think of this as a “recipe” for someone else to follow. Continue to refine your “recipe” until it is clear and deterministically solves the problem. Be sure to include the steps for prompting for input, performing calculations, and displaying output.
You should attempt to solve the problem by hand first (using a calculator as needed) to work out what the answer should be for a few sets of inputs.
Type these steps and calculations into a document (i.e., Word, text, or PDF) that will be submitted along with your source code. Note that if you do any work by hand, images (such as pictures) may be used, but they must be clear and easily readable. This document shall contain both the algorithm and any supporting hand-calculations you used in verifying your results.

TESTING:

Test your program to check that it operates as desired with a variety of inputs. Then, compare the answers your code gives with the ones you get from hand calculations.

SUBMISSION:

- Your program will be graded based largely upon whether it works correctly on the CSE machines, so you should make sure your program compiles and SAMPLE OUTPUTs on the CSE machines.

- Your program will also be graded based upon your program style. This means that you should use comments (as directed), meaningful variable names, and a consistent indentation style as recommended in the textbook and in class.

- We will be using an electronic homework submission on Canvas to make sure that all students hand in their programming projects on time. You will submit both (1) the program source code file and (2) the algorithm design document to the **Project 2** dropbox on Canvas by the due date and time.

- Projects are meant to be problem-solving exercises and are designed to help you practice your coding on larger projects with various pieces of functionality.

- If you choose to work alone, the coding should be primarily your sole work.

- If you choose to work in a group, you are allowed to get assistance from group members when working on these assignments. However, each student is required to report the name(s) of the students they worked with on the assignment. The maximum size of the group is 4 students, you can of course form a smaller group.

- Cheating for these assignments is defined as copying from a fellow student outside your working group, or copying from the web, or any other sources outside the class. You should not copy someone else's code or let a classmate examine your code if you have not identified as working in a group for your homework.

- As a safety precaution, do not edit your program (using vim or nano) after you have submitted your program where you might accidentally re-save the program, causing the timestamp on your file to be later than the due date. If you want to look (or work on it) after submitting, make a copy of your submission and work on that copy. Should there be any issues with your submission, this timestamp on your code on the CSE machines will be used to validate when the program was completed.

SAMPLE OUTPUT 1 – Checking Account, Set PIN, Deposit, Details, Exit

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
```

```
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
============================================================
              Bank Account Simulator (Console)
============================================================

Enter account holder name: Alex Johnson
Enter initial balance: $1000

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Alex Johnson
Account Type:   Checking
Balance:        $1000.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 4

Set PIN (4-6 digits): 1234
Confirm PIN: 1234
PIN set.

------------------------------------------------------------
Account Holder: Alex Johnson
Account Type:   Checking
Balance:        $1000.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 1
```

```
Enter PIN: 1234
Enter deposit amount: $250
Enter memo: Paycheck
Deposited $250.00. New balance: $1250.00

------------------------------------------------------------
Account Holder: Alex Johnson
Account Type:   Checking
Balance:        $1250.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 3

================ Account Details ================
Holder:  Alex Johnson
Type:    Checking
Balance: $1250.00
PIN:     (set)
---------------- Recent Transactions (1) ---------------
[Deposit ] $250.00 | Paycheck
--------------------------------------------------------------------
================================================

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 5
Are you sure you want to exit? (y/n): y

Thank you for using the Bank Account Simulator. Goodbye!
```

SAMPLE OUTPUT 2 — Account Type Validation

```
$ ./a.out
+-----------------------------------------------------+
|          Computer Science and Engineering           |
|           CSCE 1030 - Computer Science I            |
|         Student Name EUID euid@my.unt.edu           |
```

```
+-----------------------------------------------+

==============================================================
              Bank Account Simulator (Console)
==============================================================

Enter account holder name: Priya K
Enter initial balance: $50

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 9
Invalid choice. Please enter 1, 2, or 3.
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 0
Invalid choice. Please enter 1, 2, or 3.
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 3

--------------------------------------------------------------
Account Holder: Priya K
Account Type:   Student
Balance:        $50.00
--------------------------------------------------------------
```

SAMPLE OUTPUT 3 — PIN Creation Errors and Wrong PIN on Action

```
$ ./a.out
+-----------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-----------------------------------------------+
==============================================================
              Bank Account Simulator (Console)
==============================================================

Enter account holder name: Sam T
```

```
Enter initial balance: $75

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 2

------------------------------------------------------------
Account Holder: Sam T
Account Type:   Savings
Balance:        $75.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 1

No PIN set. Create one to continue.
Set PIN (4-6 digits): 12
Invalid PIN. Enter 4-6 digits: 12345
Confirm PIN: 99999
PIN mismatch.

Set PIN (4-6 digits): 6789
Confirm PIN: 6789
PIN set.

Enter deposit amount: $40
Enter memo: Gift
Deposited $40.00. New balance: $115.00

------------------------------------------------------------
Account Holder: Sam T
Account Type:   Savings
Balance:        $115.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
```

```
Select an option (1-6): 2

Enter PIN: 0000
Incorrect PIN.
Withdrawal canceled.
```

SAMPLE OUTPUT 4 — Student Account Overdraft Denied

```
$ ./a.out
+-----------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-----------------------------------------------+
============================================================
              Bank Account Simulator (Console)
============================================================

Enter account holder name: Jordan A
Enter initial balance: $150

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 3

------------------------------------------------------------
Account Holder: Jordan A
Account Type:   Student
Balance:        $150.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 4

Set PIN (4-6 digits): 4321
Confirm PIN: 4321
PIN set.

Select an option (1-6): 2
```

```
Enter PIN: 4321
Enter withdrawal amount: $200
Enter memo: Groceries
Withdrawal denied.
```

SAMPLE OUTPUT 5 — Checking Overdraft With Fee, Then Details

```
$ ./a.out
+-----------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-----------------------------------------------+
============================================================
              Bank Account Simulator (Console)
============================================================

Enter account holder name: Marco R
Enter initial balance: $25

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Marco R
Account Type:   Checking
Balance:        $25.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 4

Set PIN (4-6 digits): 4321
Confirm PIN: 4321
PIN set.

Select an option (1-6): 2
```

```
Enter PIN: 4321
Enter withdrawal amount: $30
Enter memo: Late-night snack
Overdraft! Fee $35.00 applied. New balance: $-40.00

Select an option (1-6): 3

================ Account Details ================
Holder:  Marco R
Type:    Checking
Balance: $-40.00
PIN:     (set)
---------------- Recent Transactions (2) ---------------
[  Fee  ] $35.00 | Overdraft Fee
[Withdraw] $30.00 | Late-night snack
-------------------------------------------------------------------
================================================
```

SAMPLE OUTPUT 6 — View Transactions By Type (Deposits then No Fees)

```
$ ./a.out
+-------------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-------------------------------------------------+
==============================================================
              Bank Account Simulator (Console)
==============================================================

Enter account holder name: Alex Johnson
Enter initial balance: $1000

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

--------------------------------------------------------------
Account Holder: Alex Johnson
Account Type:   Checking
Balance:        $1000.00
--------------------------------------------------------------
```

```
1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 4

Set PIN (4-6 digits): 1234
Confirm PIN: 1234
PIN set.

Select an option (1-6): 1

Enter PIN: 1234
Enter deposit amount: $250
Enter memo: Paycheck
Deposited $250.00. New balance: $1250.00

Select an option (1-6): 6

Show which transactions? (D=Deposit, W=Withdraw, F=Fee): d
---------------- Transactions of type Deposit ----------------
[Deposit ] $250.00 | Paycheck
-------------------------------------------------------------------

Select an option (1-6): 6

Show which transactions? (D=Deposit, W=Withdraw, F=Fee): F
(No transactions of that type)
-------------------------------------------------------------------
```

SAMPLE OUTPUT 7 — Menu Validation (Invalid then Valid)

```
$ ./a.out
+-----------------------------------------------+
|          Computer Science and Engineering       |
|           CSCE 1030 - Computer Science I        |
|         Student Name EUID euid@my.unt.edu       |
+-----------------------------------------------+
==============================================================
               Bank Account Simulator (Console)
==============================================================

Enter account holder name: Alex Johnson
Enter initial balance: $1250
```

```
Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Alex Johnson
Account Type:   Checking
Balance:        $1250.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 9
Invalid selection. Please enter 1-6.
Select an option (1-6): 0
Invalid selection. Please enter 1-6.
Select an option (1-6): 3

================ Account Details ================
Holder:  Alex Johnson
Type:    Checking
Balance: $1250.00
PIN:     (set)
---------------- Recent Transactions (1) ---------------
[Deposit ] $250.00 | Paycheck
-----------------------------------------------------------------
================================================
```

SAMPLE OUTPUT 8 — Multi-word Memo Capture

```
$ ./a.out
+--------------------------------------------------+
|          Computer Science and Engineering        |
|           CSCE 1030 - Computer Science I         |
|         Student Name EUID euid@my.unt.edu        |
+--------------------------------------------------+
============================================================
              Bank Account Simulator (Console)
============================================================
```

```
Enter account holder name: Kim P
Enter initial balance: $-40

Choose account type:
1) Checking
2) Savings
3) Student
Enter (1-3): 1

------------------------------------------------------------
Account Holder: Kim P
Account Type:   Checking
Balance:        $-40.00
------------------------------------------------------------

1) Deposit (PIN required)
2) Withdraw (PIN required)
3) Show Account Details
4) Set/Change PIN (4-6 digits)
5) Exit
6) View Transactions by Type
Select an option (1-6): 4

Set PIN (4-6 digits): 2468
Confirm PIN: 2468
PIN set.

Select an option (1-6): 1

Enter PIN: 2468
Enter deposit amount: $50
Enter memo: Transfer from savings to cover overdraft
Deposited $50.00. New balance: $10.00
```